Real-Time American Sign Language (ASL) Fingerspelling Translator

IEEE AI & CV Camp - Final Project Documentation

Omar Bahaa

April 13, 2026

Abstract

This project presents a real-time American Sign Language (ASL) fingerspelling translator powered by Deep Learning and Computer Vision. Utilizing the YOLOv8 architecture, the system translates static ASL hand gestures into text in real-time via a webcam. To ensure high accuracy, a custom Graphical User Interface (GUI) was developed using Tkinter, featuring advanced debouncing logic, snapshot loading mechanics, and auto-spacing. The model achieved a highly robust Mean Average Precision (mAP50) of 96.2%.

1 Clear Project Description

Communication barriers exist for deaf and hard-of-hearing individuals when interacting with those who do not understand sign language. The goal of this project is to bridge this gap using Artificial Intelligence. This application allows users to sign ASL alphabet letters in front of a camera, and the AI translates these gestures into a readable text format. The final product is a complete software application featuring a user-friendly interface, real-time video feedback, an alphabet reference guide, and the ability to export translated sentences to a text file.

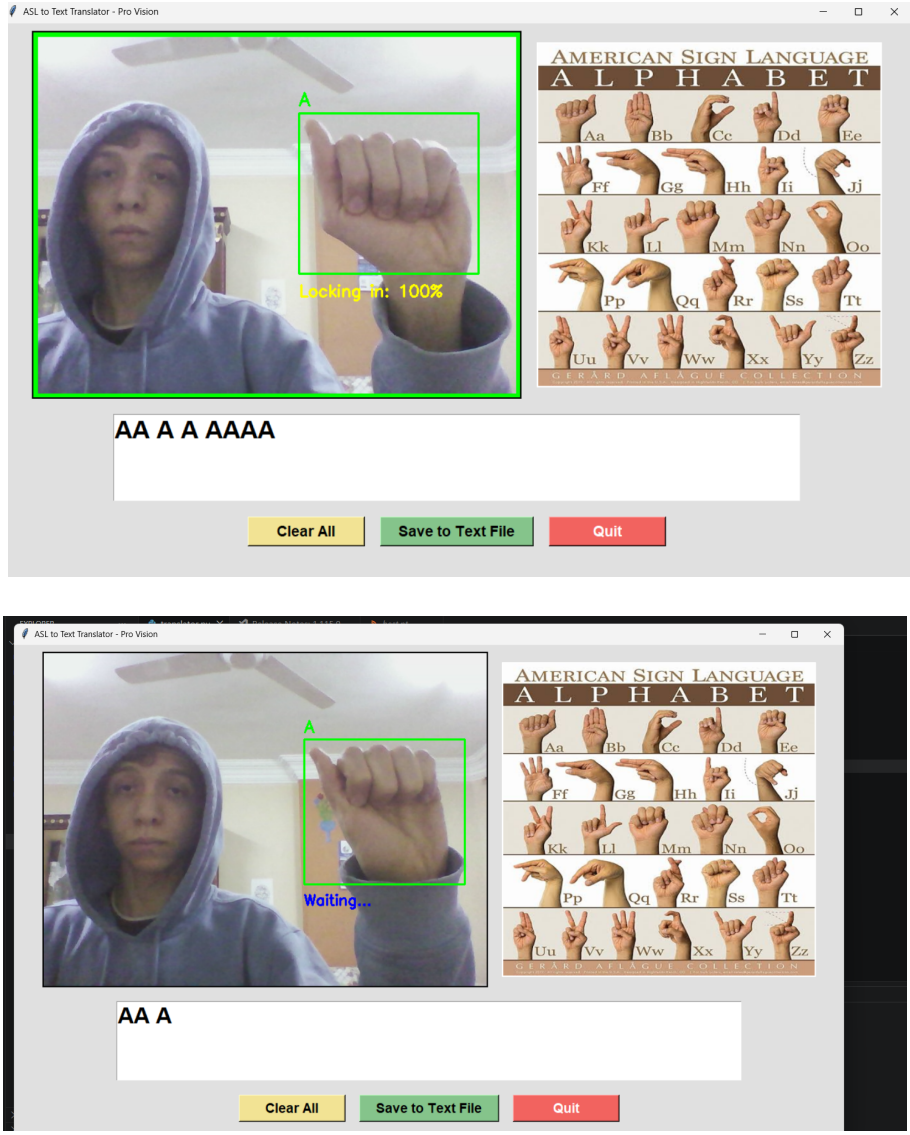

Figure 1: Proof of custom Snapshot Debouncing Logic. **Top:** The user holds the sign steady, the loading bar hits 100%, and the screen flashes green to "lock in" the translation. **Bottom:** The system immediately enters a blue "Waiting..." cooldown phase to prevent accidental double-typing before the user forms the next letter.

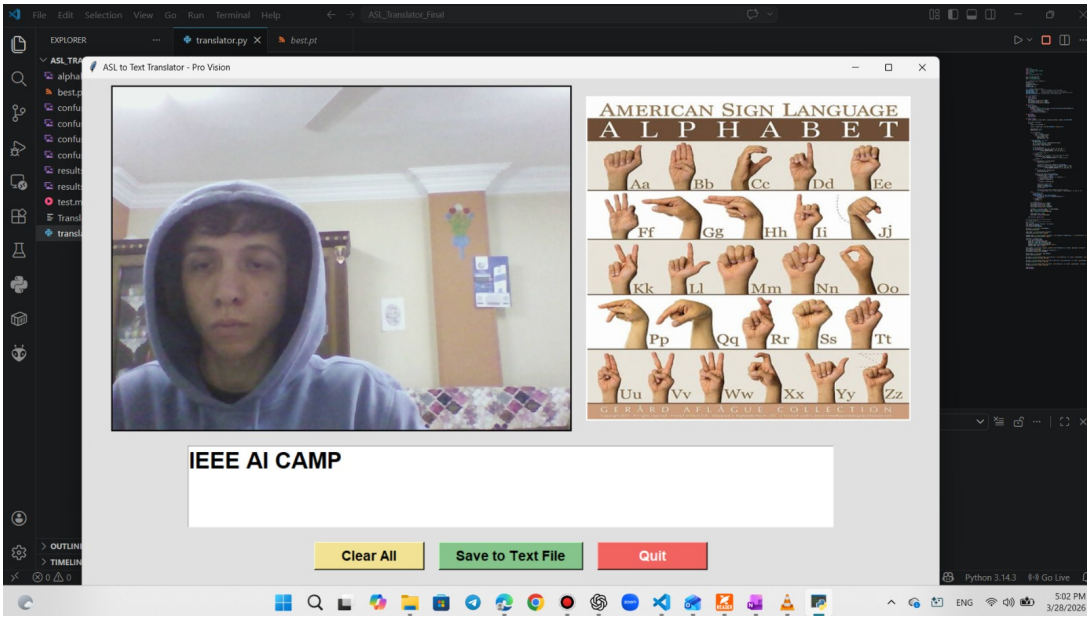

Figure 2: The custom Tkinter GUI featuring the live webcam feed, text translation box, and ASL alphabet cheat sheet. As a proof of concept, notice the system successfully translated and auto-spaced the phrase "IEEE AI CAMP" in real-time.

2 Technology Stack

- **Core Machine Learning:** Ultralytics YOLOv8, PyTorch
- **Computer Vision:** OpenCV (`cv2`)
- **User Interface:** Python Tkinter, Pillow (PIL)
- **Training Environment:** Google Colab (Tesla T4 GPU)
- **Data Management:** Roboflow

3 Step-by-Step Explanation

The development of this project followed a standard Machine Learning pipeline:

1. **Data Collection & Augmentation:** A comprehensive dataset of ASL letters was sourced. Data augmentation techniques were applied to simulate real-world webcam conditions.
2. **Cloud Training Environment:** The model was trained on a cloud-based GPU using Google Colab for faster processing.
3. **Two-Phase Training:** The model initially trained for 30 epochs to establish base feature extraction, followed by a Phase 2 training of 70 epochs to master background noise and motion blur.

4. **GUI & Logic Engineering:** A custom Python Tkinter GUI was built. A "Snapshot Loading Bar" logic was implemented to prevent rapid misclassifications (flickering), forcing the AI to lock in on steady hand signs.

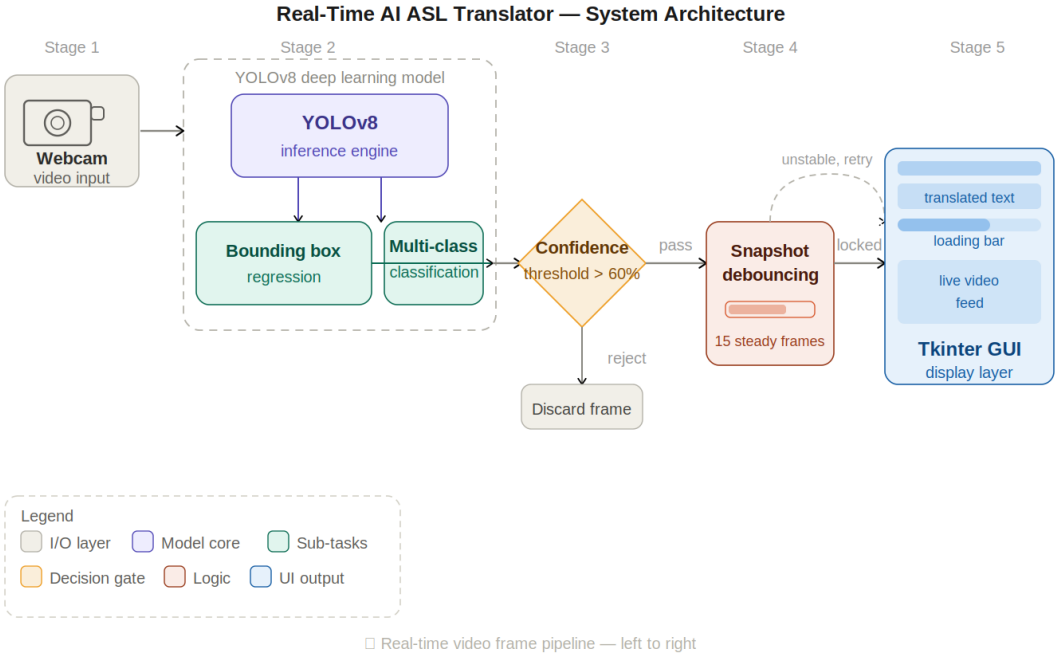

Figure 3: System Architecture: From Webcam Input to Deep Learning Inference and GUI Output.

4 Model Details & Dataset Configuration

4.1 Dataset Details

The model was trained on a custom ASL Alphabet dataset formatted via Roboflow.

- **Total Images:** Over 600 annotated images.
- **Data Split:** 80% Training, 15% Validation, 5% Testing.
- **Resolution:** Images were uniformly resized to 640x640 pixels to match the optimal input dimensions for the YOLOv8 architecture.
- **Classes:** 26 distinct classes representing the English alphabet (A-Z).

4.2 YOLOv8 Architecture

The core intelligence of this project relies on **YOLOv8**, a state-of-the-art Convolutional Neural Network (CNN). Unlike traditional image classifiers, YOLOv8 solves two Machine Learning problems simultaneously:

- **Bounding Box Regression:** The model outputs continuous mathematical coordinates to draw a precise bounding box around the tracked hand.
- **Multi-Class Classification:** Simultaneously, the model calculates probability scores to classify the contents of that bounding box into one of 26 discrete categories.

5 Challenges and Engineering Solutions

During the development of this real-time system, several technical challenges were encountered and resolved:

- **Motion Blur & Detection Flickering:** Initially, the model attempted to classify every single frame (30 FPS). As the hand moved between letters, the motion blur caused the model to output random, incorrect letters.
  Solution: I engineered a custom "Snapshot Debouncing Logic" in Python. The system now requires the YOLOv8 model to maintain a high-confidence prediction for a consecutive number of frames. A visual "Loading Bar" GUI was added to guide the user to hold still, turning a continuous video stream into high-quality static snapshots.
- **Cloud Environment Volatility:** Training deep learning models on Google Colab introduces the risk of runtime disconnections, which wipes temporary local files and datasets.
  Solution: I implemented a strict model-checkpointing routine. By downloading the `best.pt` weights locally after Phase 1, I was able to successfully restore the model's "brain" and resume Phase 2 training seamlessly.

6 Results and Analysis

After completing 100 total epochs of training, the model demonstrated exceptional performance metrics:

- **Overall Accuracy:** The model achieved a **mAP50 of 96.2%** across all 26 classes.
- **Precision & Recall:** The model achieved a precision of 92.8% and a recall of 88.7%, indicating very few false positives.
- **Class-Specific Performance:** Several complex letters (A, B, F, H, M) reached a near-perfect mAP50 of 99.5%.

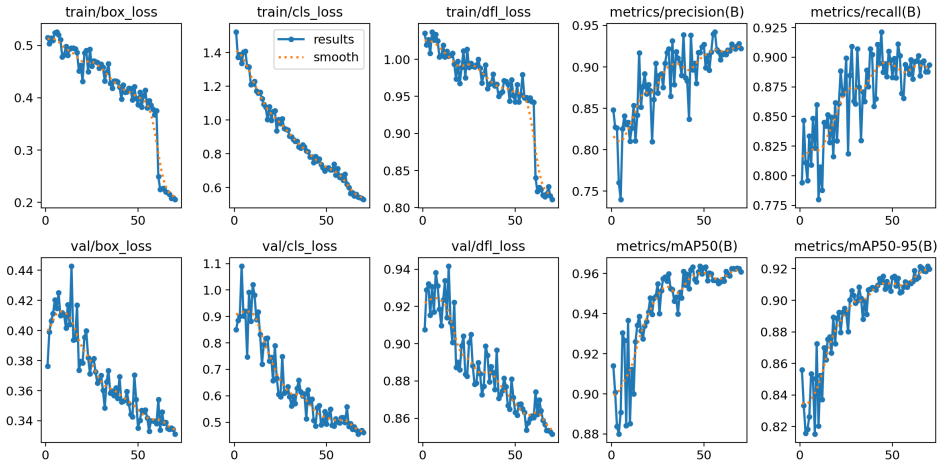

Figure 4: Training metrics showing significant loss reduction over 100 epochs.

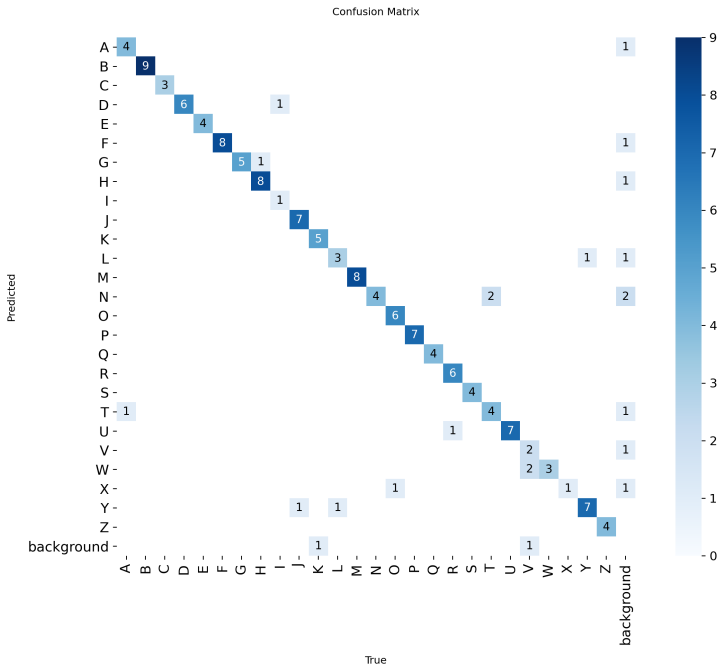

Figure 5: Confusion Matrix demonstrating high classification accuracy across all 26 letters.

7 Limitations & Future Work

While highly accurate, this model functions as an *ASL Fingerspelling Translator* (classifying static spatial features). True Sign Language Recognition requires understanding dynamic motion, time-sequence data, and facial expressions to translate full words (e.g., "Hello", "Thank you").

For future iterations, I plan to transition from a purely spatial model (YOLOv8) to a temporal model. By extracting skeletal keypoints using MediaPipe and feeding those

coordinates into a Long Short-Term Memory (LSTM) network, the system will be able to recognize fluid, moving sign language gestures over continuous video frames.

8 Conclusion

This project successfully demonstrates the application of Deep Learning to solve a real-world accessibility problem. By combining a highly optimized YOLOv8 model with robust application logic and a professional GUI, the resulting software provides a reliable, user-friendly, and real-time ASL translation experience.